

FITBUDDY

AI Fitness Plan Generator

Project Documentation

Project Overview

FitBuddy is a web-based application that uses generative AI to create personalized 7-day workout plans and practical nutrition or recovery tips from user information such as age, weight, fitness goal, and preferred workout intensity.

The system combines a FastAPI backend, Jinja2-rendered HTML pages, SQLite/SQLAlchemy persistence, and Gemini-based AI generation. Users can also submit feedback so an existing plan can be revised.

Core Technologies: Python • FastAPI • Jinja2 • SQLite • SQLAlchemy • Gemini API

Documentation format: Condensed from the supplied project documentation into exactly 12 pages.

1. Project Description and Objectives

FitBuddy simplifies the creation of structured fitness routines by collecting a small set of user inputs and passing them to an AI-powered planning layer. The application returns a day-by-day plan and a complementary nutrition or recovery tip.

Objectives

- Generate a structured 7-day workout plan from a user's goal and preferred intensity.
- Provide concise nutrition or recovery guidance related to the selected goal.
- Allow users to revise a plan through natural-language feedback.
- Store user information and both original and updated plans for later access.
- Provide an administrative view for reviewing registered users and their plans.
- Offer both an interactive HTML interface and API endpoints.

Main User Scenarios

- **Scenario 1 – Plan generation:** The user enters name, user ID, age, weight, goal, and intensity and receives a personalized weekly plan.
- **Scenario 2 – Feedback update:** The user requests changes such as more cardio or additional rest; the AI produces a revised plan.
- **Scenario 3 – Nutrition/recovery:** A concise tip is generated to complement the user's selected goal.
- **Scenario 4 – Administration:** An administrator/coach can view users and compare original and updated plans.

2. Technical Architecture

FitBuddy follows a modular web architecture in which the frontend collects information, FastAPI manages application logic, Gemini provides AI generation, and SQLite stores persistent data.

Layer	Technology / Role
Frontend	HTML + Jinja2 templates for forms, results, feedback, and admin views.
Backend	FastAPI routes, validation, request processing, and template rendering.
AI Layer	Gemini models for workout generation, nutrition tips, and feedback-based plan updates.
Database	SQLite with SQLAlchemy ORM for user and plan persistence.
Server	Uvicorn ASGI server for local development and testing.

High-Level Flow

- User → HTML/Jinja2 form → FastAPI route
- FastAPI → validation → Gemini workout/nutrition functions
- AI response → database storage → result template
- Feedback → original plan + feedback → AI revision → updated plan
- Admin request → database records → all_users.html

3. Development Workflow

The project is organized into five major milestones that move from model selection through local deployment.

Milestone	Focus	Activities
Milestone 1	Model selection and architecture	Research AI model requirements, define the application architecture, and prepare the development environment.
Milestone 2	Core functionality	Implement workout generation, nutrition tips, feedback updates, and user/plan storage.
Milestone 3	routes.py development	Connect frontend forms, AI functions, validation, database operations, and response templates.
Milestone 4	Frontend development	Build responsive Jinja2 pages for user input, results, feedback, and administration.
Milestone 5	Deployment	Prepare the virtual environment, configure environment variables, run Uvicorn, and verify the application locally.

Development Environment

- Python and pip
- Virtual environment using venv
- FastAPI and Uvicorn
- Jinja2 templates
- SQLAlchemy and SQLite
- Google Gemini SDK
- Environment variables for the Gemini API key

4. AI Model Integration

The supplied documentation describes a Pro model for structured workout generation and plan revision, and a Flash model for fast nutrition/recovery tips. The implementation keeps model names configurable through environment variables.

Workout Plan Generation

- Function: **generate_workout_gemini()**.
- Inputs include username, age, weight, goal, and intensity.
- The prompt requests exactly seven days with warm-up, main workout, exercises, sets/repetitions or duration, rest guidance, and cooldown/recovery.
- The response is returned as plain text and displayed on the result page.

Nutrition Tip Generation

- Function: **generate_nutrition_tip_with_flash()**.
- Generates a concise 2–4 sentence practical tip related to the user's goal.
- The implementation avoids extreme diets, supplement prescriptions, and medical treatment advice.

Feedback-Based Revision

- Function: **update_workout_plan()**.
- The original 7-day plan and the user's feedback are supplied to the AI.
- The revised response keeps seven days, includes recovery/rest, and meaningfully applies the feedback.

Safety note: FitBuddy is documented as general wellness guidance and not medical advice. The application prompts also instruct the AI not to diagnose conditions or prescribe treatment.

5. Backend and Database

FastAPI provides the application backend and connects the web interface to the AI and persistence layers. SQLAlchemy is used as the ORM over SQLite.

Database Model

- User ID – unique identifier for the user.
- Username, age, weight, goal, and workout intensity.
- Original workout plan generated for the user.
- Updated workout plan when feedback has been applied.
- Nutrition tip associated with the user's goal.
- Feedback submitted by the user.
- Created and updated timestamps.

Validation

- User input is validated through Pydantic schemas.
- Supported goals in the supplied implementation include weight loss, muscle gain, general wellness, and flexibility.
- Supported intensity values are low, medium, and high.
- User ID and feedback are validated before database operations and AI revision.

Persistence Flow

- Create/update user record → generate plan and nutrition tip → save to SQLite.
- Submit feedback → retrieve original plan → generate revised plan → save updated plan and feedback.
- Admin view → retrieve user records in reverse creation order.

6. FastAPI Routes and Application Logic

The routes layer acts as the operational bridge between forms, AI functions, database operations, and Jinja2 templates.

Route	Purpose	Output
/	Displays the main input form.	index.html
/generate-workout	Validates user input, calls workout and nutrition generators, and saves the result.	result.html
/submit-feedback	Retrieves the user's original plan and creates an updated plan from feedback.	Updated result.html
/view-all-users	Loads registered users and their original/updated plans.	all_users.html
/delete-user/{user_id}	Deletes a selected user record.	Redirect to admin view
/api/users	Returns user summary data through an API endpoint.	JSON
/api/users/{user_id}	Returns detailed information for one user.	JSON
/health	Checks service availability.	Health status

Route Processing Sequence

- Receive form request → validate with Pydantic → call AI function → save data → render template.
- For feedback, retrieve the existing user first so the original plan can be preserved.

7. Frontend Development

The frontend uses HTML, CSS, JavaScript, and Jinja2 templates. The supplied project documentation identifies three main pages: `index.html`, `result.html`, and `all_users.html`.

`index.html` – User Input

- Collects username, user ID, age, weight, fitness goal, and workout intensity.
- Uses a clear form layout with a fitness-oriented visual design.
- Submits the form to the FastAPI `/generate-workout` endpoint.

`result.html` – Personalized Plan

- Displays the user's submitted profile information.
- Shows the generated 7-day workout plan and nutrition tip.
- Provides a feedback form for later plan revision.

`all_users.html` – Administration

- Displays users in a structured table.
- Shows original and updated plans using expandable plan sections.
- Provides a delete action for individual user records.

Frontend Support

- Jinja2 binds backend data into HTML using template expressions.
- CSS provides responsive layout, spacing, buttons, cards, and readable result blocks.
- `app.js` provides UI behavior such as a generating/loading state and automatic textarea resizing.

8. User Interface – Home Page

The home page is the primary entry point. It asks the user for the minimum information needed to generate a personalized plan.

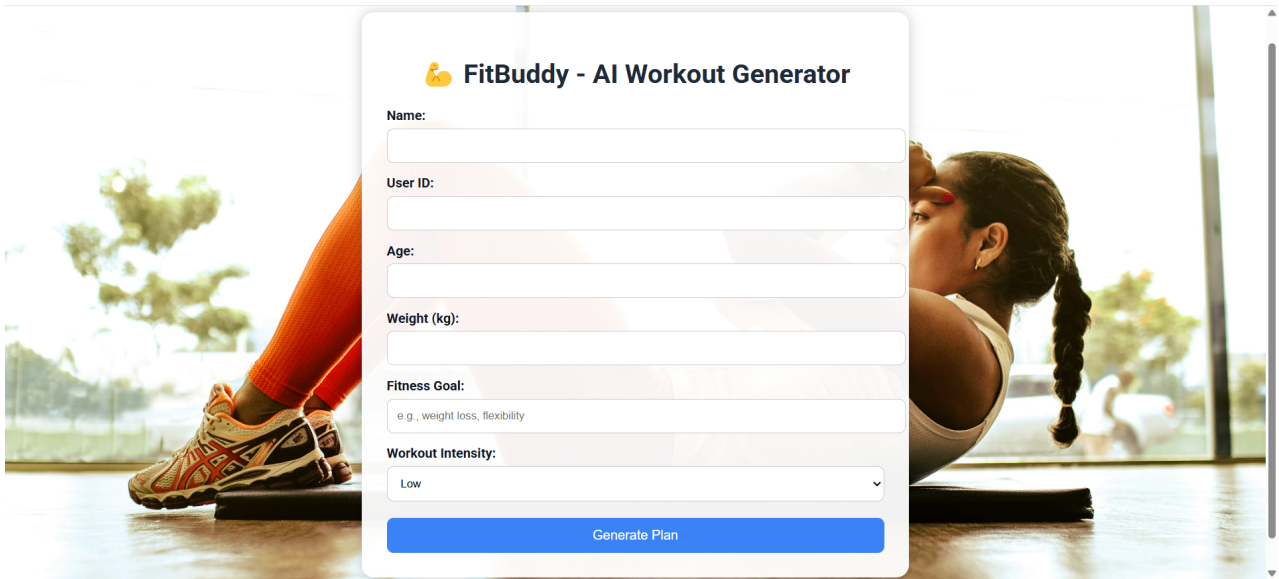
The image shows a web form titled "FitBuddy - AI Workout Generator" overlaid on a background image of a person's legs in orange leggings and sneakers, and a woman's head in profile. The form contains the following fields: "Name:" (text input), "User ID:" (text input), "Age:" (text input), "Weight (kg):" (text input), "Fitness Goal:" (text input with placeholder "e.g., weight loss, flexibility"), and "Workout Intensity:" (dropdown menu with "Low" selected). A blue "Generate Plan" button is at the bottom of the form.

Figure 1. FitBuddy home page used for collecting user details and workout preferences.

The interface includes fields for name, user ID, age, weight, fitness goal, and workout intensity, followed by a Generate Plan action. The visual design uses a fitness-themed background and a centered form to keep the workflow straightforward.

User Flow

- Enter personal details.
- Choose a fitness goal and intensity.
- Submit the form.
- FastAPI validates the data and invokes the AI generation functions.
- The generated result is rendered on the personalized plan page.

9. Feedback and Adaptive Planning

FitBuddy includes a feedback loop so the user can request changes to the current plan. This makes the generated routine adaptable instead of being limited to the first AI response.

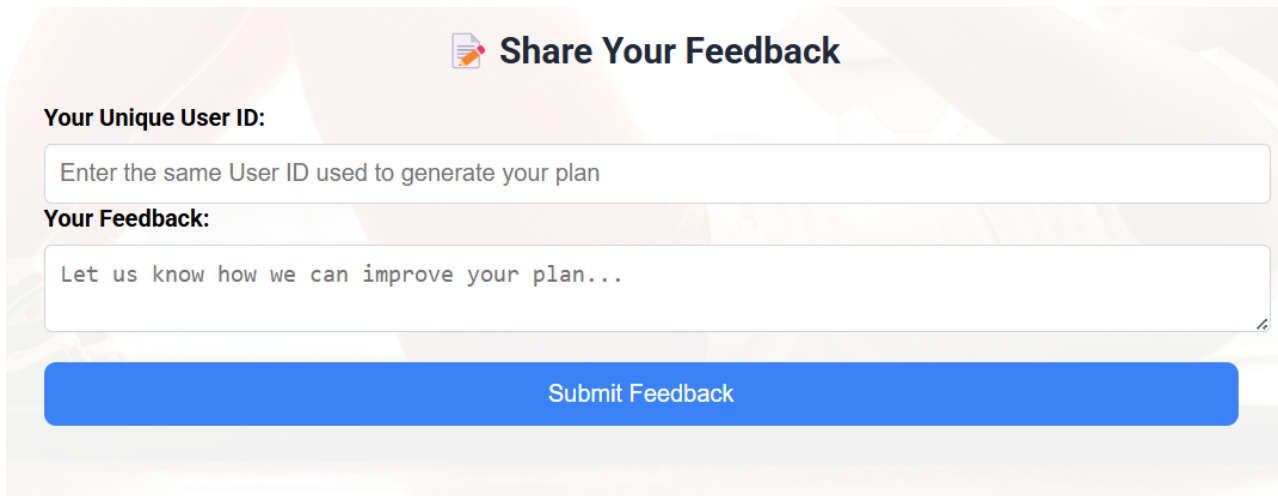
The image shows a web interface for providing feedback. At the top, there is a header with a document icon and the text "Share Your Feedback". Below this, there are two input fields. The first is labeled "Your Unique User ID:" and contains the placeholder text "Enter the same User ID used to generate your plan". The second is labeled "Your Feedback:" and contains the placeholder text "Let us know how we can improve your plan...". At the bottom of the form is a large blue button with the text "Submit Feedback". The background of the form is a light gray with a faint image of a person's legs.

Figure 2. Feedback interface used to submit a user ID and requested plan changes.

Feedback Process

- The user enters the same unique User ID used during plan generation.
- The user describes the desired change, such as adding cardio or including more recovery.
- FastAPI retrieves the stored original plan.
- The original plan and feedback are sent to the plan-update function.
- The revised plan is stored separately so the original plan remains available.
- A confirmation message is returned after the update.

This approach preserves the original plan while providing an updated version, supporting comparison and continued personalization.

10. Setup, Testing and Local Deployment

The supplied documentation describes local execution with a Python virtual environment and Uvicorn.

Environment Setup

- Create a virtual environment with **python -m venv venv**.
- Activate it using the platform-specific activation command.
- Install dependencies from the project's requirements file.
- Configure the Gemini API key through an environment variable or .env file.

Run the Application

uvicorn main:app --reload

The local application is accessed through **http://127.0.0.1:8000**. FastAPI's interactive documentation is available through **/docs**.

Testing Checklist

- Open the home page and verify the form loads.
- Submit valid user information and confirm a plan is generated.
- Verify that the nutrition/recovery tip is displayed.
- Submit feedback and confirm that an updated plan is returned.
- Open the admin view and verify stored user records.
- Use the health endpoint to confirm that the service is running.

Security Note

Do not place a real API key in the documentation. Store credentials in environment variables or a protected .env file and exclude that file from public source control.

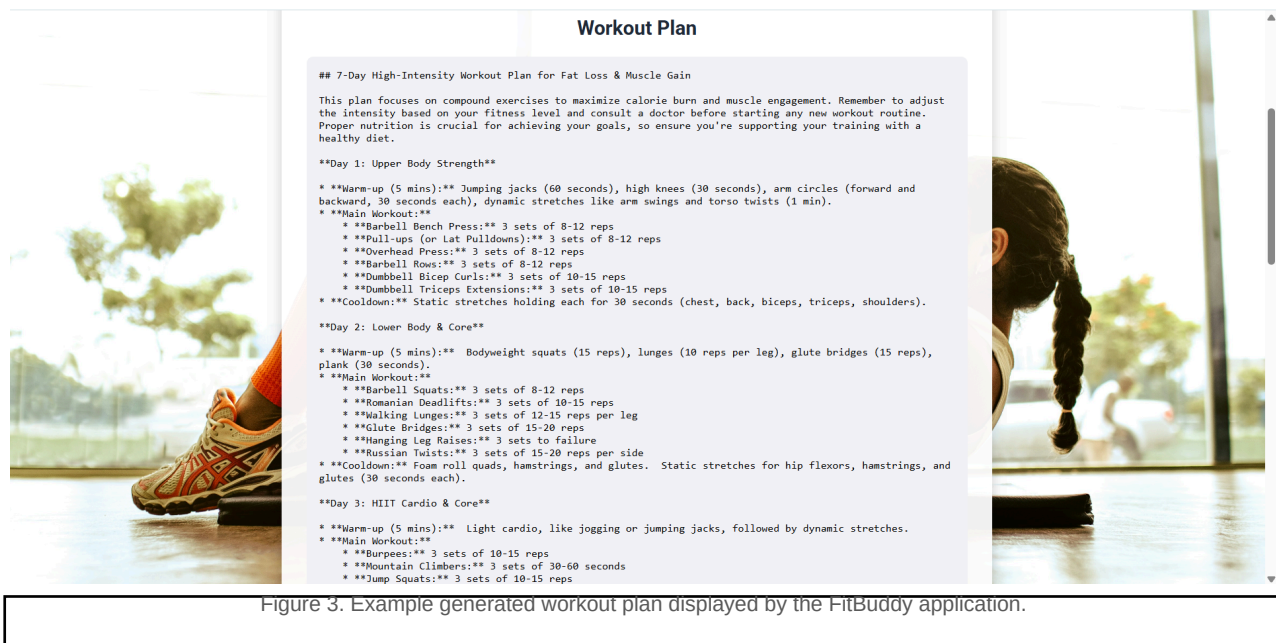


Figure 3. Example generated workout plan displayed by the FitBuddy application.

11. Conclusion and Future Scope

Conclusion

FitBuddy demonstrates how generative AI can be integrated into a practical web application for personalized fitness planning. The project combines FastAPI backend logic, Jinja2 templates, SQLite/SQLAlchemy persistence, and Gemini-powered generation to produce structured 7-day plans and supporting nutrition or recovery guidance.

A key feature is the feedback mechanism: users can submit comments about their current plan and receive an updated version while the original plan remains stored. The administrative view also provides a structured way to inspect users and their plan history.

Future Scope

- Add authentication and role-based access for users, coaches, and administrators.
- Add progress tracking, workout completion history, and visual analytics.
- Add richer exercise media such as instructional images or videos.
- Add optional integrations with wearable or activity data where appropriate.
- Improve deployment readiness for a cloud environment.
- Add automated testing for routes, validation, database operations, and AI-response handling.

Final Project Summary

Component	Purpose
FastAPI	Backend routing and application logic
Gemini	Workout, nutrition, and plan-revision generation
Jinja2 + HTML/CSS/JS	Interactive web interface
SQLite + SQLAlchemy	Persistent user and plan storage
Uvicorn	Local ASGI application server

Source basis: the uploaded FitBuddy project documentation and the accompanying project-code text. The content has been condensed and reorganized into a 12-page documentation format, with two project-interface images retained from the supplied document.